

Library

class **Consist**

#include <[DCCEXLoco.h](#)>

DEPRECATED! Class to create a software consist of one or more [ConsistLoco](#) objects.

As of version 1.3.0, all [Consist](#) and related classes and methods are being deprecated in favour of the new command station consists.

Public Functions

Consist()

DEPRECATED Constructor.

void setName(const char *name)

DEPRECATED Set consist name.

Parameters: **name** – Name to set for the consist

const char *getName()

DEPRECATED Get consist name.

Returns: Current name of the consist

void addLoco(Loco *loco, Facing facing)

DEPRECATED Add a loco to the consist using a [Loco](#) object.

Parameters: • **loco** – Pointer to a loco object
 • **facing** – Direction the loco is facing (FacingForward|FacingReversed)

void addLoco(int address, Facing facing)

DEPRECATED Add a loco to the consist using a DCC address.

Parameters: • **address** – DCC address of the loco to add
 • **facing** – Direction the loco is facing (FacingForward|FacingReversed)

void removeLoco(Loco *loco)

DEPRECATED Remove a loco from the consist - [Loco](#) objects with LocoSourceEntry will also be deleted.

Parameters: **loco** – Pointer to a loco object to remove

void removeAllLocos()



DEPRECATED Remove all locos from a consist - [Loco](#) objects with LocoSourceEntry will also be deleted.

void setLocoFacing([Loco](#) *loco, [Facing](#) facing)

DEPRECATED Update the direction of a loco in the consist.

Parameters:

- **loco** – Pointer to the loco object to update
- **facing** – Direction to set it to (FacingForward|FacingReversed)

int getLocoCount()

DEPRECATED Get the count of locos in the consist.

Returns: Count of locos

bool inConsist([Loco](#) *loco)

DEPRECATED Check if the provided loco is in the consist.

Parameters: **loco** – Pointer to the loco object to check
Returns: true|false

bool inConsist(int address)

DEPRECATED Check if the loco with the provided address is in the consist.

Parameters: **address** – DCC address of loco to check
Returns: true|false

int getSpeed()

DEPRECATED Get consist speed - obtained from first linked loco.

Returns: Current speed (0 - 126)

[Direction](#) getDirection()

DEPRECATED Get consist direction - obtained from first linked loco.

Returns: Current direction (Forward|Reverse)

[ConsistLoco](#) *getFirst()

DEPRECATED Get the first loco in the consist.

Returns: Pointer to the first [ConsistLoco](#) object

[ConsistLoco](#) *getByAddress(int address)

DEPRECATED Get the loco in the consist with the specified address.

Parameters: **address** – DCC address of loco to retrieve
Returns: Pointer to the first [ConsistLoco](#) object

~Consist()

DEPRECATED Destructor for a [Consist](#).

Private Functions

void _addLocoToConsist([ConsistLoco](#) *consistLoco)

Private Members

char *_name

int _locoCount

[ConsistLoco](#) *_first

[class](#) **ConsistLoco**

#include <[DCCExLoco.h](#)>

DEPRECATED! Class to add an additional attribute to a [Loco](#) object to specify the direction it is facing in a consist.

As of version 1.3.0, all [Consist](#) and related classes and methods are being deprecated in favour of the new command station consists.

Public Functions

ConsistLoco([Loco](#) *loco, [Facing](#) facing)

DEPRECATED Constructor.

- Parameters:**
- **loco** – Pointer to the [Loco](#) object to add
 - **facing** – Direction loco is facing in the consist (FacingForward|FacingReversed)

[Loco](#) *getLoco()

DEPRECATED Get the associated [Loco](#) object for this consist entry.

Returns: Pointer to the [Loco](#) object

void setFacing([Facing](#) facing)

DEPRECATED Set which way the loco is facing in the consist (FacingForward, FacingReversed)

Parameters: **facing** – FacingForward|FacingReversed

[Facing](#) getFacing()

DEPRECATED Get which way the loco is facing in the consist (FacingForward, FacingReversed)

Returns: FacingForward|FacingReversed

ConsistLoco *getNext()

DEPRECATED Get the next consist loco object.

Returns: Pointer to the next [ConsistLoco](#) object

void setNext(ConsistLoco *consistLoco)

DEPRECATED Set the next consist loco object.

Parameters: **consistLoco** – Pointer to the [ConsistLoco](#) object

~ConsistLoco()

DEPRECATED Destructor for a [ConsistLoco](#).

Private Members

Loco *_loco

Facing_facing

ConsistLoco *_next

Friends

friend class Consist

class CConsist

#include <[DCCEXCConsist.h](#)>

Class to assist managing command station consists.

In order for the command station to accept a [CConsist](#), at least two locos are required. Each member loco is a [CConsistMember](#) object, enabling them to be operated in reverse compared to normal direction of travel. Each [CConsist](#) instance is in a linked list accessible via [CConsist::getFirst\(\)](#). It is recommended to always create and delete [CConsist](#) objects using the methods provided in [DCCEXProtocol](#) class to ensure your command station is updated appropriately.

Public Functions

CConsist(bool replicateFunctions = false)

Construct a new [CConsist](#) object.

Parameters: **replicateFunctions** – Replicate function control to all member locos (default false)

CConsist *getNext()

Get the Next object.

Returns: CConsist* Pointer to the next [CConsist](#) object

void addMember(int address, bool reversed)

Add a loco by address to the consist.

This method attempts to find an existing [Loco](#) object and will use that instead of creating a new object.

- Parameters:**
- **address** – DCC address of the loco to be added
 - **reversed** – True if loco is reversed to normal direction of travel

void removeMember(int address)

Remove a loco by address from the consist.

- Parameters:** **address** – DCC address of the loco to be removed

void removeAllMembers()

Remove all [CSConsistMember](#) objects from this consist.

[CSConsistMember](#) *getFirstMember()

Get the First Member object.

- Returns:** [CSConsistMember](#)* Pointer to the first [CSConsistMember](#) object

[CSConsistMember](#) *getMember(int address)

Get the Member [Loco](#) object.

- Parameters:** **address** – DCC address of the loco associated with the member
Returns: [CSConsistMember](#)* Pointer to the [CSConsistMember](#) object

bool isInConsist(int address)

Check by address if a loco is in this consist.

- Parameters:** **address** – DCC address of the loco
Returns: true If loco address is in consist
Returns: false If not

bool isReversed(int address)

Check by address if the loco is reversed in this consist.

- Parameters:** **address** – DCC address of the loco
Returns: true If loco address is in consist and reversed
Returns: false If loco address is not in consist or is not reversed

bool isValid()

Check if this is a valid consist with more than 1 member.

- Returns:** true Valid [CSConsist](#)
Returns: false Invalid (less than 2 members)

int getMemberCount()

Get the number of members in the [CSConsist](#).

Returns: int Member count

bool getAlwaysReplicateFunctions()

Get the Always Replicate Functions setting.

Returns: true If static/global flag enabled

Returns: false If not

void setReplicateFunctions(bool replicate)

Set the Replicate Functions flag for this [CSConsist](#).

Parameters: replicate – True if function control should apply to all members

bool getReplicateFunctions()

Get the Replicate Functions flag for this [CSConsist](#).

Returns: true True if function control should apply to all members

Returns: false False if not

int getSpeed()

Get the speed from the lead [Loco](#).

Returns: int Speed of the lead [Loco](#), 0 always if no [Loco](#) object matches

Direction getDirection()

Get the direction from the lead [Loco](#).

Returns: Direction Direction of the lead [Loco](#), Forward always if no [Loco](#) object matches

~CSConsist()

Destroy the [CSConsist](#) object.

Public Static Functions

static [CSConsist](#) *getFirst()

Get the First object.

Returns: CSConsist* Pointer to the first [CSConsist](#) object

static void clearCSConsists()

Clear all CSConsists from the list.

static [CSConsist](#) *getLeadLocoCSConsist(int address)

Get the [CSConsist](#) the provided address is lead loco of.

Parameters: **address** – DCC address of the lead loco to check for

Returns: **CSConsist*** Pointer to the [CSConsist](#) object, or nullptr if none found

```
static CSConsist *getMemberCSConsist(int address)
```

Get the [CSConsist](#) the provided address is a member of.

Parameters: **address** – DCC address of the member loco to check for

Returns: **CSConsist*** Pointer to the [CSConsist](#) object, or nullptr if none found

```
static void setAlwaysReplicateFunctions(bool replicate)
```

Set the default behaviour for function replication for all new [CSConsist](#) objects.

Call this method once before creating any [CSConsist](#) objects to ensure all inherit this behaviour.

Parameters: **replicate** – True if all newly created [CSConsist](#) objects should replicate functions

Private Members

```
bool _replicateFunctions
```

```
CSConsistMember *_firstMember
```

```
CSConsist *_next
```

```
int _memberCount
```

Private Static Attributes

```
static CSConsist *_first = nullptr
```

```
static bool _alwaysReplicateFunctions = false
```

[struct](#) CSConsistMember

```
#include <DCCEXCSConsist.h>
```

Structure for a [CSConsistMember](#).

Public Functions

```
inline CSConsistMember(int address, bool reversed)
```

Construct a new [CSConsistMember](#) object.

Parameters:

- **address** – DCC address of the member
- **reversed** – True if reversed to normal direction of travel

Public Members

`uint16_t address`

`uint16_t reversed`

`CConsistMember *next`

`class DCCEXInbound`

`#include <DCCEXInbound.h>`

Inbound DCC-EX command parser class to parse commands and provide interpreted parameters.

Public Static Functions

`static void setup(int16_t maxParameterValues)`

Setup parser once with enough space to handle the maximum number of parameters expected from the command station.

Parameters: `maxParameterValues` – Maximum parameter values to accommodate

`static void cleanup()`

Cleanup parser.

`static bool parse(char *command)`

Pass in a command string to parse.

Parameters: `command` – Char array of command to parse

Returns: True if parsed ok, false if badly terminated command or too many parameters

`static byte getOpcode()`

Gets the DCC-EX OPCODE of the parsed command (the first char after the <)

`static int16_t getParameterCount()`

Gets number of parameters detected after OPCODE <JR 1 2 3> is 4 parameters!

Returns: Number of parameters

`static int32_t getNumber(int16_t parameterNumber)`

Gets a numeric parameter (or hashed keyword) from parsed command.

Returns: The numeric parameter

`static bool isTextParameter(int16_t parameterNumber)`

Checks if a parameter is actually text rather than numeric.

Parameters: **parameterNumber** – The number of the parameter to check

Returns: true|false

`static char *getTextParameter(int16_t parameterNumber)`

Gets address of text type parameter. does not create permanent copy.

Parameters: **parameterNumber** – The number of the parameter to retrieve

Returns: Char array of text (use once and discard)

`static char *copyTextParameter(int16_t parameterNumber)`

gets address of a heap copy of text type parameter.

Parameters: **parameterNumber** –

Returns:

`static void dump(Print*)`

dump list of parameters obtained

Parameters: **out** – Address of output e.g. &Serial

Private Static Functions

`static bool _isTextInternal(int16_t n)`

Private Static Attributes

`static int16_t _maxParams = 0`

`static int16_t _parameterCount = 0`

`static byte _opcode = 0`

`static int32_t *_parameterValues = nullptr`

`static char *_cmdBuffer = nullptr`

`class DCCExProtocol`

`#include <DCCExProtocol.h>`

Main class for the [DCCExProtocol](#) library.

Public Functions

`DCCExProtocol(int maxCmdBuffer = 500, int maxCommandParams = 50, unsigned long userChangeDelay = 100)`

Constructor for the [DCCExProtocol](#) object.

Parameters: • **maxCmdBuffer** – Optional - maximum number of bytes for the command buffer (default 500)

- **maxCommandParams** – Optional - maximum number of parameters to parse via the [DCCEXInbound](#) parser (default 50)
- **userChangeDelay** – Optional - time in ms between sending throttle changes (default 100)

~DCCEXProtocol()

Destructor for the [DCCEXProtocol](#) object.

void setDelegate([DCCEXProtocolDelegate](#) *delegate)

Set the delegate object for callbacks.

Parameters: **delegate** –

void setLogStream(Stream *console)

Set the stream object for console output.

Parameters: **console** –

void enableHeartbeat(unsigned long heartbeatDelay = 60000)

Enable heartbeat if required - can help WiFi connections that drop out.

Parameters: **heartbeatDelay** – Time in milliseconds between heartbeats - defaults to one minute (60000ms)

void connect(Stream *stream)

Connect the stream object to interact with DCC-EX.

Parameters: **stream** –

void disconnect()

DEPRECATED - Does nothing, retained for backwards compatibility only.

Will be removed in 2.0.0.

void check()

Check for incoming DCC-EX broadcasts/responses and parse them.

void sendCommand(const char *cmd)

allows sending of an arbitray command

Parameters: **cmd** – Command to send

void getLists(bool rosterRequired = true, bool turnoutListRequired = true, bool routeListRequired = true, bool turntableListRequired = true)

Request DCC-EX object lists (Roster, Turnouts, Routes, Turntables)

Parameters: • **rosterRequired** – Request the roster list (true|false)

- **turnoutListRequired** – Request the turnout list (true|false)
- **routeListRequired** – Request the route list (true|false)
- **turntableListRequired** – Request the turntable list (true|false)

bool receivedLists()

Check if all lists have been received (roster, routes, turnouts, turntables)

Returns: true|false

void requestServerVersion()

Request server version information.

bool receivedVersion()

Check if server version has been received.

Returns:

int getMajorVersion()

Retrieve the major version of EX-CommandStation.

Returns: Major version number eg. 5.y.z

int getMinorVersion()

Retrieve the minor version of EX-Commandstation.

Returns: Minor version number eg. x.0.z

int getPatchVersion()

Retrieve the patch version of EX-CommandStation.

Returns: Patch version number eg. x.y.7

unsigned long getLastServerResponseTime()

Retrieve the last time the server responded.

Returns: Last response time in milliseconds (from millis())

void clearAllLists()

Clear roster, turnout, turntable, and route lists.

void refreshAllLists()

Clear roster, turnout, turntable, and route lists and request new ones.

void setDebug(bool debug)

Set the Debug flag.

Parameters: **debug** – True to send output commands to the console (defaults to False, no output)

void setThrottle(Loco *loco, int speed, Direction direction)

Set the provided loco to the specified speed and direction.

Parameters:

- **loco** – Pointer to a [Loco](#) object
- **speed** – Speed (0 - 126)
- **direction** – Direction (Forward|Reverse)

void setThrottle(Consist *consist, int speed, Direction direction)

DEPRECATED Set all locos in the provided consist to the specified speed and direction.

Will be removed in 2.0.0, use [setThrottle\(CSConsist *consist, int speed, Direction direction\)](#)

Parameters:

- **consist** – Pointer to a consist object
- **speed** – Speed (0 - 126)
- **direction** – Direction (Forward|Reverse) - reverse facing locos will be adjusted automatically

void setThrottle(CSConsist *csConsist, int speed, Direction direction)

Set the provided command station consist to the specified speed and direction.

If this consist does not exist, it will be created first, and if it is not valid, this command will be ignored.

Parameters:

- **csConsist** – Pointer to the [CSConsist](#) object
- **speed** – Speed (0 - 126)
- **direction** – Direction (Forward|Reverse)

void functionOn(Loco *loco, int function)

Turn the specified function on for the provided loco.

Parameters:

- **loco** – Pointer to a loco object
- **function** – Function number (0 - 27)

void functionOn(Consist *consist, int function)

DEPRECATED Turn the specified function on for the provided consist.

Will be removed in 2.0.0, use [functionOn\(CSConsist *csConsist, int function\)](#)

Parameters:

- **consist** – Pointer to a consist object
- **function** – Function number (0 - 27)

void functionOn(CSConsist *csConsist, int function)

Turn the specified function on for the provided [CSConsist](#).

Parameters:

- **csConsist** – Pointer to the [CSConsist](#) object
- **function** – Function number (0 - 27)

void functionOff([Loco](#) *loco, int function)

Turn the specified function off for the provided loco.

- Parameters:
- **loco** – Pointer to a loco object
 - **function** – Function number (0 - 27)

void functionOff([Consist](#) *consist, int function)

DEPRECATED Turn the specified function off for the provided consist.

Will be removed in 2.0.0, use [functionOff\(CSConsist *csConsist, int function\)](#)

- Parameters:
- **consist** – Pointer to a consist object
 - **function** – Function number (0 - 27)

void functionOff([CSConsist](#) *csConsist, int function)

Turn the specified function off for the provided [CSConsist](#).

- Parameters:
- **csConsist** – Pointer to the [CSConsist](#) object
 - **function** – Function number (0 - 27)

bool isFunctionOn([Loco](#) *loco, int function)

Test if the specified function for the provided loco is on.

- Parameters:
- **loco** – Pointer to a loco object
 - **function** – Function number to test (0 - 27)

Returns: true = on, false = off

bool isFunctionOn([Consist](#) *consist, int function)

DEPRECATED Test if the specified function for the provided consist is on (Checks first loco)

Will be removed in 2.0.0, use [isFunctionOn\(CSConsist *csConsist, int function\)](#)

- Parameters:
- **consist** – Pointer to a consist object
 - **function** – Function number to test (0 - 27)

Returns: true = on, false = off

bool isFunctionOn([CSConsist](#) *csConsist, int function)

Test if the specified function for the provided [CSConsist](#) is on (checks first [Loco](#))

- Parameters:
- **csConsist** – Pointer to the [CSConsist](#) object
 - **function** – Function number to test (0 - 27)

Returns: true Function on

Returns: false Function off, or [CSConsist](#) object is invalid

void requestLocoUpdate(int address)

Explicitly request an update for the specified loco.

- Parameters:
- **address** – DCC address of the loco

void readLoco()

Initiate reading a loco address from the programming track, response will be a delegate notification.

void emergencyStop()

Initiate an emergency stop.

int getRosterCount()

Get the number of roster entries.

Returns: Number of roster entries received

bool receivedRoster()

Check if roster has been received.

Returns: true|false

Loco *findLocoInRoster(int address)

Search for the specified DCC address in the roster.

Parameters: **address** – DCC address to search for

Returns: Pointer to the [Loco](#) object

void clearRoster()

Clear the roster.

void clearLocalLocos()

Clear the list of local locos.

void refreshRoster()

Clear the roster and request again.

void requestCSConsists()

Request the list of CSConsists from the command station, will create [CSConsist](#) objects.

CSConsist *createCSConsist(int leadLoco, bool reversed = *false*, bool replicateFunctions = *false*)

Create a [CSConsist](#).

Parameters:

- **leadLoco** – DCC address of the lead loco
- **reversed** – True if loco reversed to normal direction of travel (sending Forward will cause it to reverse)

Returns: CSConsist* Pointer to the created [CSConsist](#) object

bool addCSConsistMember([CSConsist](#) *csConsist, int address, bool reversed = *false*)

Add a member to the [CSConsist](#).

Parameters:

- **address** – DCC address of the member
- **reversed** – True if loco reversed to normal direction of travel (sending Forward will cause it to reverse)

Returns: bool True if added successfully, otherwise false

[CSConsist](#) *getCSConsistByLeadLoco(int address)

Get a [CSConsist](#) by the lead loco DCC address.

Parameters: **address** – DCC address of the lead loco

Returns: [CSConsist](#)* Pointer to the [CSConsist](#) object the loco is lead for, or nullptr if none exist

[CSConsist](#) *getCSConsistByLeadLoco([Loco](#) *loco)

Get a [CSConsist](#) by the lead [Loco](#) object.

Parameters: **address** – Pointer to the lead [Loco](#) object

Returns: [CSConsist](#)* Pointer to the [CSConsist](#) object the loco is lead for, or nullptr if none exist

[CSConsist](#) *getCSConsistByMemberLoco(int address)

Get a [CSConsist](#) by a member DCC address.

Parameters: **address** – DCC address of the member

Returns: [CSConsist](#)* Pointer to the [CSConsist](#) object the loco is a member of, or nullptr if none exist

[CSConsist](#) *getCSConsistByMemberLoco([Loco](#) *loco)

Get a [CSConsist](#) by a member [Loco](#) object.

Parameters: **address** – Pointer to the member [Loco](#) object

Returns: [CSConsist](#)* Pointer to the [CSConsist](#) object the loco is a member of, or nullptr if none exist

bool removeCSConsistMember([CSConsist](#) *csConsist, int address)

Remove a member from the [CSConsist](#).

Parameters: **address** – DCC address of the member

Returns: bool True if removed successfully, otherwise false

void deleteCSConsist(int leadLoco)

Delete the [CSConsist](#) using the lead loco address.

Parameters: **address** – DCC address of the lead loco

void deleteCSConsist([CSConsist](#) *csConsist)

Delete the [CSConsist](#).

Parameters: **csConsist** – Pointer to the [CSConsist](#) object to delete

void clearCSConsists()

Clears all [CSConsist](#) objects.

void setMomentumAlgorithm([MomentumAlgorithm](#) algorithm)

Set the Momentum Algorithm.

Parameters: **algorithm** – MomentumAlgorithm (Linear, Power)

void setDefaultMomentum(int momentum)

Set the Default Momentum for accelerating/braking.

Parameters: **momentum** – Momentum value

void setDefaultMomentum(int accelerating, int braking)

Set the Default Momentum for separate accelerating/braking.

Parameters:

- **accelerating** – Accelerating momentum value
- **braking** – Braking momentum value

void setMomentum(int address, int momentum)

Set momentum for the specified loco/DC track.

Parameters:

- **address** – DCC address
- **momentum** – Momentum value

void setMomentum([Loco](#) *loco, int momentum)

Set the Momentum for the specified [Loco](#).

Parameters:

- **loco** – Pointer to a [Loco](#) object
- **momentum** – Momentum value

void setMomentum(int address, int accelerating, int braking)

Set the Momentum for separate accelerating/braking.

Parameters:

- **address** – DCC address
- **accelerating** – Accelerating momentum value
- **braking** – Braking momentum value

void setMomentum([Loco](#) *loco, int accelerating, int braking)

Set the Momentum for separate accelerating/braking.

Parameters:

- **loco** – Pointer to a [Loco](#) object
- **accelerating** – Accelerating momentum value

- **braking** – Braking momentum value

int getTurnoutCount()

Get the number of turnouts.

Returns: Number of turnouts received

bool receivedTurnoutList()

Check if turnout list has been received.

Returns: true|false

Turnout *getTurnoutById(int turnoutId)

Retrieve a turnout/point object by its ID.

Parameters: **turnoutId** – ID of the turnout/point

Returns: The turnout/point object

void closeTurnout(int turnoutId)

Close the specified turnout/point.

Parameters: **turnoutId** – ID of the turnout/point

void throwTurnout(int turnoutId)

Throw the specified turnout/point.

Parameters: **turnoutId** – ID of the turnout/point

void toggleTurnout(int turnoutId)

Toggle the specified turnout/point (if closed, will throw, and vice versa)

Parameters: **turnoutId** – ID of the turnout/point

void clearTurnoutList()

Clear the list of turnouts.

void refreshTurnoutList()

Clear the list of turnouts and request again.

int getRouteCount()

Get the number of route entries.

Returns: Number of routes received

bool receivedRouteList()

Check if route list has been received.

Returns: true|false

void startRoute(int routeId)

Start a route/automation.

Parameters: routeId – ID of the route/automation to start

void handOffLoco(int locoAddress, int automationId)

Hand off a [Loco](#) to an Automation - note you must retrieve the ROUTE list to use this.

Parameters:

- locoAddress – DCC address of the [Loco](#) to hand off
- automationId – ID of the automation to start, must be RouteType::RouteTypeAutomation

void pauseRoutes()

Pause all routes/automations.

void resumeRoutes()

Resume all routes/automations.

void clearRouteList()

Clear all routes.

void refreshRouteList()

Clear all routes and request a new list.

int getTurntableCount()

Get the number of turntable entries.

Returns: Number of turntables received

bool receivedTurntableList()

Check if turntable list has been received.

Returns: true|false

[Turntable](#) ***getTurntableById(int turntableId)**

Retrieve a turntable object by its ID.

Parameters: turntableId – ID of the turntable

Returns: The turntable object

void rotateTurntable(int turntableId, int position, int activity = 0)

Rotate a turntable object.

Parameters:

- turntableId – ID of the turntable

- **position** – Position index to rotate to
- **activity** – Optional activity for EX-Turntable objects only

void clearTurntableList()

Clear all turntables.

void refreshTurntableList()

Clear all turntables and request a new list.

void powerOn()

Global track power on command.

void powerOff()

Global track power off command.

void powerMainOn()

Main track power on command.

void powerMainOff()

Main track power off command.

void powerProgOn()

Prog track power on command.

void powerProgOff()

Prog track power off command.

void joinProg()

Join Prog track to Main.

void powerTrackOn(char track)

Turn power on for the specified track.

Parameters: **track** – Track name (A - H)

void powerTrackOff(char track)

Turn power off for the specified track.

Parameters: **track** – Track name (A - H)

void setTrackType(char track, [TrackManagerMode](#) type, int address)

set track type for the specified track

- Parameters:
- **track** – Track name (A - H)
 - **type** – Track type (MAIN, PROG, DC, DCX, NONE)
 - **address** – dcc address for DC and DCX (Required, but ignored if not DC or DCX)

void requestTrackCurrentGauges()

Request the current limit set for each track.

void requestTrackCurrents()

Request the latest current value for each track.

void activateAccessory(int accessoryAddress, int accessorySubAddr)

Activate DCC accessory at the specified address and subaddress.

- Parameters:
- **accessoryAddress** – Address of the DCC accessory
 - **accessorySubAddr** – Subaddress of the DCC accessory

void deactivateAccessory(int accessoryAddress, int accessorySubAddr)

Deactivate DCC accessory at the specified address and subaddress.

- Parameters:
- **accessoryAddress** – Address of the DCC accessory
 - **accessorySubAddr** – Subaddress of the DCC accessory

void activateLinearAccessory(int linearAddress)

Activate DCC accessory at the specified linear address.

- Parameters:
- **linearAddress** – Linear address of the DCC accessory

void deactivateLinearAccessory(int linearAddress)

Deactivate DCC accessory at the specified linear address.

- Parameters:
- **linearAddress** – Linear address of the DCC accessory

void getNumberSupportedLocos()

Request the number of supported cabs(locos)

void readCV(int cv)

Read the value of the provided CV from the [Loco](#) on the programming track.

- Parameters:
- **cv** – CV number to read the value of

void validateCV(int cv, int value)

Validate the provided value is stored in the provided CV.

- Parameters:
- **cv** – CV number to validate the value of
 - **value** – Value to validate

void validateCVBit(int cv, int bit, int value)

Validate the provided bit is set to the specified value for the provided CV.

- Parameters:**
- **cv** – CV number to validate the bit of
 - **bit** – Bit for the CV to validate
 - **value** – Value to validate (0|1)

void writeLocoAddress(int address)

Write [Loco](#) address to the [Loco](#) on the programming track.

- Parameters:**
- **address** – DCC address to write

void writeCV(int cv, int value)

Write the provided value to the specified CV on the programming track.

- Parameters:**
- **cv** – CV number to write to
 - **value** – Value to write to the CV

void writeCVBit(int cv, int bit, int value)

Write the provided value to the specified bit and CV on the programming track. Note that the response is in legacy DCC++ formatting and is not supported by the [DCCExInbound](#) parser. Therefore, the throttle will not receive a response. We suggest writing full CVs.

- Parameters:**
- **cv** – CV number to write to
 - **bit** – Bit for the CV to write
 - **value** – Value to write (0|1)

void writeCVOnMain(int address, int cv, int value)

Write the provided value to the specified [Loco](#) address and CV on the main track.

- Parameters:**
- **address** – DCC address of the [Loco](#)
 - **cv** – CV number to write to
 - **value** – Value to write to the CV

void writeCVBitOnMain(int address, int cv, int bit, int value)

Write the provided value to the specified [Loco](#), CV, and bit on the main track.

- Parameters:**
- **address** – DCC address of the [Loco](#)
 - **cv** – CV number to write to
 - **bit** – Bit for the CV to write
 - **value** – Value to write (0|1)

void setFastClock(int minutes, int speedFactor)

Set the fast clock time and speed factor.

- Parameters:**
- **minutes** – Time from midnight in minutes (eg. 60 = 1am)

- **speedFactor** – Speed factor multiplier (eg. 4 = 1 minute every 15 seconds)

void requestFastClockTime()

Request the current fast clock time.

void requestJMRI SensorList()

Request the list of JMRI sensors.

Public Members

Loco *roster = nullptr

Linked list of **Loco** objects to form the roster, call roster->getFirst()

Turnout *turnouts = nullptr

Linked list of **Turnout** objects to form the turnout list, call turnouts->getFirst()

Route *routes = nullptr

Linked list of **Route** objects to form the list of routes and automations, call routes->getFirst()

Turntable *turntables = nullptr

Linked list of **Turntable** objects to form the list of turntables, call turntables->getFirst()

CSConsist *csConsists = nullptr

Linked list of **CSConsist** objects to make these accessible via the **DCCEXProtocol** class, call csConsists->getFirst()

Public Static Functions

static const char *getLibraryVersion()

Get the current **DCCEXProtocol** library version.

Returns: const char* Version string

Private Functions

void _init()

void _sendCommand()

void _processCommand()

void _processServerDescription()

```
void _processMessage()
```

```
void _processScreenUpdate()
```

```
void _sendHeartbeat()
```

```
void _processLocoBroadcast()
```

```
int _getValidFunctionMap(int functionMap)
```

```
int _getSpeedFromSpeedByte(int speedByte)
```

```
Direction _getDirectionFromSpeedByte(int speedByte)
```

```
void _setLocos(Loco *firstLoco)
```

```
void _updateLocos(Loco *firstLoco, int address, int speedByte, Direction direction, int functionMap)
```

```
void _processReadResponse()
```

```
void _processPendingUserChanges()
```

```
void _processCSConsist()
```

```
void _buildCSConsist(CSConsist *csConsist, int memberCount)
```

```
void _sendCreateCSConsist(CSConsist *csConsist)
```

```
void _sendDeleteCSConsist(CSConsist *csConsist)
```

```
void _setCSConsistMemberFunction(CSConsistMember *member, int function, bool state)
```

```
void _getRoster()
```

```
void _processRosterList()
```

```
void _requestRosterEntry(int address)
```

```
void _processRosterEntry()
```

```
void _getTurnouts()
```

`void _processTurnoutList()`

`void _requestTurnoutEntry(int id)`

`void _processTurnoutEntry()`

`void _processTurnoutBroadcast()`

`void _getRoutes()`

`void _processRouteList()`

`void _requestRouteEntry(int id)`

`void _processRouteEntry()`

`void _getTurntables()`

`void _processTurntableList()`

`void _requestTurntableEntry(int id)`

`void _processTurntableEntry()`

`void _requestTurntableIndexEntry(int id)`

`void _processTurntableIndexEntry()`

`void _processTurntableBroadcast()`

`void _processTrackPower()`

`void _processTrackType()`

`void _processTrackCurrentGauges()`

`void _processTrackCurrents()`

`void _processValidateCVResponse()`

`void _processValidateCVBitResponse()`

`void _processWriteLocoResponse()`

void _processWriteCVResponse()

void _processSetFastClock()

void _processFastClockTime()

void _processJMRIISensorBroadcast(byte opcode)

void _cmdStart(char opcode = '\0')

Start the command with "<" and the OPCODE if required.

Parameters: **opcode** – Single character OPCODE to append

void _cmdAppend(const char *s)

Append a string to the command.

Parameters: **s** – String to append

void _cmdAppend(int n)

Append an integer value to the command.

Parameters: **n** – Integer to append

void _cmdAppend(char c)

Append a single char to the command.

Parameters: **c** – Char to append

void _cmdSend()

Append the closing ">" and call _sendCommand()

void _sendOpcode(char opcode)

Formatter for opcode only outbound commands.

Parameters: **opcode** – OPCODE to send

void _sendOneParam(char opcode, char param)

Formatter for opcode and one parameter.

Parameters: • **opcode** – OPCODE to send
 • **param** – Single char parameter to send

void _sendOneParam(char opcode, const char *param)

Formatter for opcode and one parameter.

Parameters: • **opcode** – OPCODE to send

- **param** – Single string parameter to send

void _sendOneParam(char opcode, int param)

Formatter for opcode and one parameter.

- Parameters:**
- **opcode** – OPCODE to send
 - **param** – Single int parameter to send

void _sendTwoParams(char opcode, char param1, int param2)

Formatter for opcode and two params.

- Parameters:**
- **opcode** – OPCODE to send
 - **param1** – Single char parameter to send
 - **param2** – Single int parameter to send

void _sendTwoParams(char opcode, int param1, int param2)

Formatter for opcode and two params.

- Parameters:**
- **opcode** – OPCODE to send
 - **param1** – Single int parameter to send
 - **param2** – Single int parameter to send

void _sendTwoParams(char opcode, const char *param1, int param2)

Formatter for opcode and two params.

- Parameters:**
- **opcode** – OPCODE to send
 - **param1** – Single string parameter to send
 - **param2** – Single int parameter to send

void _sendTwoParams(char opcode, int param1, char param2)

Formatter for opcode and two params.

- Parameters:**
- **opcode** – OPCODE to send
 - **param1** – Single int parameter to send
 - **param2** – Single char parameter to send

void _sendTwoParams(char opcode, char param1, const char *param2)

Formatter for opcode and two params.

- Parameters:**
- **opcode** – OPCODE to send
 - **param1** – Single char parameter to send
 - **param2** – Single string parameter to send

void _sendThreeParams(char opcode, const char *param1, int param2, int param3)

Formatter for opcode and three params.

- Parameters:**
- **opcode** – OPCODE to send

- **param1** – Single string parameter to send
- **param2** – Single int parameter to send
- **param3** – Single int parameter to send

void _sendThreeParams(char opcode, int param1, int param2, int param3)

Formatter for opcode and three params.

- Parameters:
- **opcode** – OPCODE to send
 - **param1** – Single int parameter to send
 - **param2** – Single int parameter to send
 - **param3** – Single int parameter to send

void _sendThreeParams(char opcode, char param1, const char *param2, int param3)

Formatter for opcode and three params.

- Parameters:
- **opcode** – OPCODE to send
 - **param1** – Single char parameter to send
 - **param2** – Single string parameter to send
 - **param3** – Single int parameter to send

void _sendThreeParams(char opcode, char param1, int param2, int param3)

Formatter for opcode and three params.

- Parameters:
- **opcode** – OPCODE to send
 - **param1** – Single char parameter to send
 - **param2** – Single int parameter to send
 - **param3** – Single int parameter to send

void _sendFourParams(char opcode, int param1, int param2, int param3, int param4)

Formatter for opcode and four params.

- Parameters:
- **opcode** – OPCODE to send
 - **param1** – Single int parameter to send
 - **param2** – Single int parameter to send
 - **param3** – Single int parameter to send
 - **param4** – Single int parameter to send

Private Members

int _rosterCount = 0

int _turnoutCount = 0

int _routeCount = 0

int _turntableCount = 0

```
int _version[3] = {}
```

```
Stream *_stream
```

```
Stream *_console
```

```
NullStream _nullStream
```

```
int _bufflen
```

```
int _maxCmdBuffer
```

```
char *_cmdBuffer
```

```
char _outboundCommand[MAX_OUTBOUND_COMMAND_LENGTH]
```

```
DCCEXProtocolDelegate *_delegate = nullptr
```

```
unsigned long _lastServerResponseTime
```

```
char _inputBuffer[512]
```

```
int _nextChar
```

```
bool _receivedVersion = false
```

```
bool _receivedLists = false
```

```
bool _rosterRequested = false
```

```
bool _receivedRoster = false
```

```
bool _turnoutListRequested = false
```

```
bool _receivedTurnoutList = false
```

```
bool _routeListRequested = false
```

```
bool _receivedRouteList = false
```

```
bool _turntableListRequested = false
```

```
bool _receivedTurntableList = false
```

`bool _enableHeartbeat`

`unsigned long _heartbeatDelay`

`unsigned long _lastHeartbeat`

`int _cmdIndex`

`unsigned long _userChangeDelay`

`unsigned long _lastUserChange`

`bool _debug = false`

***class* DCCEXProtocolDelegate**

#include <[DCCEXProtocol.h](#)>

Delegate responses and broadcast events to the client software to enable custom event handlers.

Public Functions

inline virtual void **receivedServerVersion**(int major, int minor, int patch)

Notify when the server version has been received.

- Parameters:**
- **major** – Major version of EX-CommandStation (eg. 5.0.7 returns 5)
 - **minor** – Minor version of EX-CommandStation (eg. 5.0.7 returns 0)
 - **patch** – Patch version of EX-CommandStation (eg. 5.0.7 returns 7)

inline virtual void **receivedMessage**(const char *message)

Notify when a broadcast message has been received.

Parameters: **message** – message that has been broadcast

inline virtual void **receivedRosterList**()

Notify when the roster list is received.

inline virtual void **receivedTurnoutList**()

Notify when the turnout list is received.

inline virtual void **receivedRouteList**()

Notify when the route list is received.

inline virtual void **receivedTurntableList**()

Notify when the turntable list is received.

inline virtual void **receivedLocoUpdate**(Loco *loco)

Notify when an update to a Loco object is received.

Parameters: loco – Pointer to the loco object

inline virtual void **receivedLocoBroadcast**(int address, int speed, Direction direction, int functionMap)

Notify when a Loco broadcast has been received - suitable for non-roster locos.

Parameters:

- address – DCC address of the loco
- speed – Speed as derived from the speed byte
- direction – Direction as derived from the speed byte
- functionMap – Function map

inline virtual void **receivedTrackPower**(TrackPower state)

Notify when the global track power state change is received.

Parameters: state – Power state received (PowerOff|PowerOn|PowerUnknown)

inline virtual void **receivedTrackCurrentGauge**(char track, int limit)

Notify when a track current limit value is received.

Parameters:

- track – Track (A - H)
- limit – Current limit in mA

inline virtual void **receivedTrackCurrent**(char track, int current)

Notify when a track current value is received.

Parameters:

- track – Track (A - H)
- current – Current in mA

inline virtual void **receivedIndividualTrackPower**(TrackPower state, int track)

Notify when an individual track power state change is received.

Parameters:

- state – Power state received (PowerOff|PowerOn|PowerUnknown)
- track – which track changed 65=A..72=H | 2698315=MAIN | 2788330=PROG | 2183=DC | 71999=DCX

inline virtual void **receivedTrackType**(char track, TrackManagerMode type, int address)

Notify when a track type change is received.

Parameters:

- track – track that changed
- type – type received (MAIN|PROG|DC|DCX|NONE)
- address – address received for DC and DCX (zero if other types)

inline virtual void **receivedTurnoutAction**(int turnoutId, bool thrown)

Notify when a turnout state change is received.

- Parameters:
- **turnoutId** – ID of the turnout
 - **thrown** – Whether it is thrown or not (true|false)

inline virtual void **receivedTurntableAction**(int turnoutId, int position, bool moving)

Notify when a turntable index change is received.

- Parameters:
- **turntableId** – ID of the turntable
 - **position** – Index of the position it is moving (or has moved) to
 - **moving** – Whether it is moving or not (true|false)

inline virtual void **receivedReadLoco**(int address)

Notify when a loco address is read from the programming track.

- Parameters:
- **address** – DCC address read from the programming track, or -1 for a failure to read

inline virtual void **receivedValidateCV**(int cv, int value)

Notify when a CV is read or validated from the programming track.

- Parameters:
- **cv** – CV the value has been read from
 - **value** – Value read from the CV, or -1 for a failure to read

inline virtual void **receivedValidateCVBit**(int cv, int bit, int value)

Notify when a CV bit is validated from the programming track.

- Parameters:
- **cv** – CV the bit is being validated in
 - **bit** – Bit of the CV being validated
 - **value** – Value validated from the bit, or -1 if not valid

inline virtual void **receivedWriteLoco**(int address)

Notify when a [Loco](#) address has been written on the programming track.

- Parameters:
- **address** – DCC address written to the loco, or -1 for a failure to write

inline virtual void **receivedWriteCV**(int cv, int value)

Notify when a CV is written on the programming track.

- Parameters:
- **cv** – CV being written to
 - **value** – Value written, or -1 for failure

inline virtual void **receivedScreenUpdate**(int screen, int row, const char *message)

Notify when a screen update is received.

- Parameters:
- **screen** – Screen number
 - **row** – Row number
 - **message** – Message to display on the screen/row

inline virtual void receivedCSConsist(int leadLoco, [CSConsist](#) *csConsist)

Notify when a CS consist has been received.

- Parameters:**
- **leadLoco** – DCC address of the lead loco for the consist
 - **csConsist** – Pointer to the [CSConsist](#) object controlled by the lead loco

inline virtual void receivedSetFastClock(int minutes, int speedFactor)

Notify when a fast clock time has been set.

- Parameters:**
- **minutes** – Time since midnight in minutes
 - **speedFactor** – Speed factor multiplier

inline virtual void receivedFastClockTime(int minutes)

Notify when a fast clock time has been received.

- Parameters:** **minutes** – Time in minutes

inline virtual void receivedJMRI sensorBroadcast(int id, [JMRI sensorState](#) state)

Notify when a JMRI sensor broadcast has been received.

- Parameters:**
- **id** – ID of the JMRI sensor
 - **state** – JMRI sensorState enum value

virtual ~DCCExProtocolDelegate() = default

Default destructor for [DCCExProtocolDelegate](#).

class Loco

#include <[DCCExLoco.h](#)>

Class for a [Loco](#) object representing a DCC addressed locomotive.

Public Functions

Loco(int address, [LocoSource](#) source)

Constructor.

- Parameters:**
- **address** – DCC address of loco
 - **source** – LocoSourceRoster (from roster) or LocoSourceEntry (from user input)

int **getAddress**()

Get loco address.

Returns: DCC address of loco

void **setName**(const char *name)

Set loco name.

Parameters: **name** – Name of the loco

const char *getName()

Get loco name.

Returns: Name of the loco

void setSpeed(int speed)

Set loco speed.

Parameters: **speed** – Valid speed (0 - 126)

int getSpeed()

Get loco speed.

Returns: Speed (0 - 126)

void setDirection(Direction direction)

Set loco direction (enums Forward, Reverse)

Parameters: **direction** – Direction to set (Forward|Reverse)

Direction getDirection()

Get loco direction (enums Forward, Reverse)

Returns: Current direction (Forward|Reverse)

LocoSource getSource()

Get loco source (enums LocoSourceRoster, LocoSourceEntry)

Returns: Source of loco (LocoSourceRoster|LocoSourceEntry)

void setupFunctions(const char *functionNames)

Setup functions for the loco.

Parameters: **functionNames** – Char array of function names

bool isFunctionOn(int function)

Test if function is on.

Parameters: **function** – Number of the function to test

Returns: true|false

void setFunctionStates(int functionStates)

Set function states.

Parameters: **functionStates** – Integer representing all function states

int getFunctionStates()

Get function states.

Returns: Integer representing current function states

const char *getFunctionName(int function)

Get the name/label for a function.

Parameters: **function** – Number of the function to return the name/label of

Returns: char* representing the function name/label

bool isFunctionMomentary(int function)

Get the name/label for a function.

Parameters: **function** – Number of the function to return the name/label of

Returns: char* representing the function name/label

void setNext(Loco *loco)

Set the next loco in the roster list.

Parameters: **loco** – Pointer to the next **Loco** object

Loco *getNext()

Get next **Loco** object.

Returns: Pointer to the next **Loco** object

void setUserSpeed(int speed)

Set the requested user speed and flag pending.

Parameters: **speed** – Requested speed

int getUserSpeed()

Get the current requested user speed.

Returns: int Requested speed

void setUserDirection(Direction direction)

Set the requested user direction and flag pending.

Parameters: **direction** – Requested Direction

Direction getUserDirection()

Get the current requested user direction.

Returns: Direction Requested Direction

void resetUserChangePending()

Reset the user change pending flag.

bool getUserChangePending()

Get the user change pending flag.

Returns: true If a speed or direction change is pending

Returns: false If no changes are pending

~Loco()

Destructor for the [Loco](#) object.

Public Static Functions

static [Loco](#) *getFirst()

Get first [Loco](#) object.

Returns: Pointer to the first [Loco](#) object

static [Loco](#) *getByAddress(int address)

Get [Loco](#) object by its DCC address.

Parameters: address – DCC address of the loco to get

Returns: [Loco](#) object or nullptr if it doesn't exist

static void clearRoster()

Clear all Locos from the roster.

static [Loco](#) *getFirstLocalLoco()

Get the First Local [Loco](#) object.

Returns: [Loco](#)* Pointer to the first loco with LocoSourceEntry type

static void clearLocalLocos()

Clear all local locos.

Private Members

int _address

char *_name

int _speed

[Direction](#) _direction

`LocoSource _source`

`char *_functionNames[MAX_FUNCTIONS]`

`int32_t _functionStates`

`int32_t _momentaryFlags`

`Loco *_next`

`int _userSpeed`

`Direction _userDirection`

`bool _userChangePending`

Private Static Functions

`static void _addToList(Loco **listHead, Loco *loco)`

Add the loco to a list.

- Parameters:
- `listHead` – Pointer to the list entry point to add it to
 - `loco` – Pointer to the `Loco` to add

`static Loco *_findAddressInList(Loco **listHead, int address)`

Helper method to find the `Loco` associated with the specified address in a list.

- Parameters:
- `listHead` – Pointer to the list entry point to look for the address
 - `address` – DCC address to look for

Returns: `Loco*`

`static void _removeFromList(Loco **listHead, Loco *loco)`

Method to remove this loco from the roster list.

- Parameters:
- `listHead` – Pointer to the list entry point to remove it from
 - `loco` – Pointer to the `Loco` to remove

`static void _clearList(Loco **listHead)`

Helper method to clear all locos from the provided list.

Parameters: `listHead` – Pointer to the first loco in the list

Private Static Attributes

`static Loco *_first = nullptr`

```
static Loco *_firstLocalLoco = nullptr
```

Friends

```
friend class Consist
```

class **NullStream** : **public** **Stream**

```
#include <DCCEXProtocol.h>
```

Nullstream class for initial **DCCEXProtocol** instantiation to direct streams to nothing.

Public Functions

```
inline NullStream()
```

Constructor for the **NullStream** object.

```
inline int available()
```

Dummy availability check.

Returns: Returns false (0) always

```
inline void flush()
```

Dummy flush method.

```
inline int peek()
```

Dummy peek method.

Returns: Returns -1 always

```
inline int read()
```

Dummy read method.

Returns: Returns -1 always

```
inline size_t write(uint8_t c)
```

Dummy write method for single int.

Parameters: c – Number received

Returns: Returns 1 always

```
inline size_t write(const uint8_t *buffer, size_t size)
```

Dummy write method for buffered input.

Parameters:

- **buffer** – Buffer received
- **size** – Size of buffer

Returns: Returns size of buffer always

class **Route**

#include <[DCCEXRoutes.h](#)>

Class to contain and maintain the various [Route](#) attributes and methods.

Public Functions

Route(int id)

Constructor.

Parameters: id – [Route](#) ID

int **getId**()

Get route ID.

Returns: ID of the route

void **setName**(const char *name)

Set route name.

Parameters: name – Name to set for the route

const char ***getName**()

Get route name.

Returns: Current name of the route

void **setType**([RouteType](#) type)

Set route type (A automation, R route)

Parameters: type – RouteType - RouteTypeAutomation|RouteTypeRoute

[RouteType](#) **getType**()

Get route type (A automation, R route)

Returns: RouteTypeAutomation|RouteTypeRoute

void **setNext**([Route](#) *route)

Set the next route in the list.

Parameters: route – Pointer to the next route

[Route](#) ***getNext**()

Get next [Route](#) object.

Returns: Pointer to the next [Route](#) object

~Route()

Destructor for a route.

Public Static Functions

`static Route *getFirst()`

Get first `Route` object.

Returns: Pointer to the first `Route` object

`static Route *getById(int id)`

Get route object by its ID.

Returns: Pointer to the `Route`, or nullptr if not found

`static void clearRouteList()`

Clear the list of routes.

Private Functions

`void _removeFromList(Route *route)`

Remove the route from the list.

Parameters: `route` – Pointer to the route to remove

Private Members

`int _id`

`char *_name`

`char _type`

`Route *_next`

Private Static Attributes

`static Route *_first = nullptr`

`class Turnout`

`#include <DCCEXTurnouts.h>`

Class to contain and maintain the various Turnout/Point attributes and methods.

Public Functions

`Turnout(int id, bool thrown)`

Constructor for a `Turnout` object.

Parameters:

- `id` – `Turnout` ID
- `thrown` – true (thrown)|false (closed)

void setThrown(bool thrown)

Set thrown state (true thrown, false closed)

Parameters: **thrown** – true|false

void setName(const char *_name)

Set turnout name.

Parameters: **_name** – Name to set the turnout

int getId()

Get turnout Id.

Returns: ID of the turnout

const char *getName()

Get turnout name.

Returns: Current name of the turnout

bool getThrown()

Get thrown state (true thrown, false closed)

Returns: true|false

void setNext(Turnout *turnout)

Set the next turnout in the list.

Parameters: **turnout** – Pointer to the next [Turnout](#)

Turnout *getNext()

Get next turnout object.

Returns: Pointer to the next [Turnout](#) object

~Turnout()

Destructor for a [Turnout](#).

Public Static Functions

static Turnout *getFirst()

Get first turnout object.

Returns: Pointer to the first [Turnout](#) object

static Turnout *getById(int id)

Get turnout object by turnout ID.

Parameters: **id** – ID of the turnout to retrieve

Returns: Pointer to the turnout object or nullptr if not found

static void clearTurnoutList()

Clear the list of turnouts.

Private Functions

void _removeFromList(Turnout *turnout)

Remove the turnout from the list.

Parameters: **turnout** – Pointer to the turnout to remove

Private Members

Turnout *_next

int _id

char *_name

bool _thrown

Private Static Attributes

static Turnout *_first = nullptr

class Turntable

#include <DCCEXTurntables.h>

Class to contain and maintain the various **Turntable** attributes and methods.

Public Functions

Turntable(int id)

Constructor.

Parameters: **id** – ID of the turntable

int getId()

Get turntable ID.

Returns: ID of the turntable

void setType(TurntableType type)

Set turntable type.

Parameters: **type** – TurntableTypeDCC|TurntableTypeEXTT|TurntableTypeUnknown

TurntableType **getType()**

Get turntable type.

Returns: TurntableTypeDCC|TurntableTypeEXTT|TurntableTypeUnknown

void **setIndex**(int index)

Set the current index for the turntable.

Parameters: index – ID of the index to set

int **getIndex**()

Get the current index for the turntable.

Returns: ID of the current index

void **setNumberOfIndexes**(int numberOfIndexes)

Set the number of indexes the turntable has defined (from the <JT id> command response)

Parameters: numberOfIndexes – Count of the indexes defined for the turntable including home

int **getNumberOfIndexes**()

Get the number of indexes defined for the turntable.

Returns: Count of the indexes defined

void **setName**(const char *name)

Set the turntable name.

Parameters: name – Name to set for the turntable

const char ***getName**()

Get the turntable name.

Returns: Current name of the turntable

void **setMoving**(bool moving)

Set the movement state (false stationary, true moving)

Parameters: moving – true|false

bool **isMoving**()

Get movement state (false stationary, true moving)

Returns: true|false

int **getIndexCount**()

Get the count of indexes added to the index list (counted from the <JP id> command response)

Returns: Count of indexes received for this turntable including home

void setNext(Turntable *turntable)

Set the next turntable in the list.

Parameters: turntable – Pointer to the next turntable

Turntable *getNext()

Get the next turntable object.

Returns: Pointer to the next Turntable object

void addIndex(TurntableIndex *index)

Add a turntable index object to the index list for this turntable.

Parameters: index – TurntableIndex object to add

TurntableIndex *getFirstIndex()

Get the first associated turntable index.

Returns: Pointer to the first associated TurntableIndex object

TurntableIndex *getIndexById(int id)

Get TurntableIndex object by its ID.

Parameters: id – ID of the index to retrieve

Returns: Pointer to the TurntableIndex object or nullptr if not found

~Turntable()

Destructor for a turntable.

Public Static Functions

static Turntable *getFirst()

Get the first turntable object.

Returns: Pointer to the first Turntable object

static Turntable *getById(int id)

Get a turntable object by its ID.

Parameters: id – ID of the turntable to retrieve

Returns: Pointer to the Turntable object or nullptr if not found

static void clearTurntableList()

Clear the list of turntables.

Private Functions

```
void _removeFromList(Turntable *turntable)
```

Remove the turntable from the list.

Parameters: **turntable** – Pointer to the turntable to remove

Private Members

```
int _id
```

```
TurntableType _type
```

```
int _index
```

```
int _numberOfIndexes
```

```
char *_name
```

```
bool _isMoving
```

```
int _indexCount
```

```
Turntable *_next
```

```
TurntableIndex *_firstIndex
```

Private Static Attributes

```
static Turntable *_first = nullptr
```

class TurntableIndex

```
#include <DCCEXTurntables.h>
```

Class to contain and maintain the various **Turntable** Index attributes and methods associated with a **Turntable**.

Public Functions

```
TurntableIndex(int ttId, int id, int angle, const char *name)
```

Constructor.

Parameters:

- **ttId** – ID of the turntable the index is associated with
- **id** – ID of the index
- **angle** – Angle from home for this index (0 - 3600)
- **name** – Name of the index

int `getTtId()`

Get the turntable ID.

Returns: ID of the turntable this index is associated with

int `getId()`

Get index ID (0 is always home)

Returns: ID of this index

int `getAngle()`

Get angle of the index from home.

Returns: Angle of this index from home (0 - 3600)

const char* `getName()`

Get index name.

Returns: Current name of the index

[TurntableIndex](#) *`getNextIndex()`

Get next [TurntableIndex](#) object.

Returns: Pointer to the next [TurntableIndex](#) object

~`TurntableIndex()`

Destructor for an index.

Private Members

int `_ttId`

int `_id`

int `_angle`

char* `_name`

[TurntableIndex](#) *`_nextIndex`

Friends

***friend class* Turntable**

file `DCCEXCSConsist.cpp`

`#include "DCCEXCSConsist.h"`

file DCCEXCSConsist.h

```
#include "DCCEXLoco.h"  
#include <Arduino.h>
```

file DCCEXInbound.cpp

```
#include "DCCEXInbound.h"  
#include <Arduino.h>
```

Enums

enum splitState

Values:

| **enumerator FIND_START**

| **enumerator SET_OPCODE**

| **enumerator SKIP_SPACES**

| **enumerator CHECK_SIGN**

| **enumerator BUILD_PARAM**

| **enumerator SKIPOVER_TEXT**

| **enumerator COMPLETE_i_COMMAND**

Variables

const int32_t QUOTE_FLAG = 0x77777000

const int32_t QUOTE_FLAG_AREA = 0xFFFFF000

file DCCEXInbound.h

```
#include <Arduino.h>
```

file DCCEXLoco.cpp

```
#include "DCCEXLoco.h"  
#include <Arduino.h>
```

file DCCEXLoco.h

```
#include <Arduino.h>
```

Defines

MAX_SINGLE_COMMAND_PARAM_LENGTH

Enums

enum Direction

Values:

| **enumerator** Reverse

| **enumerator** Forward

enum LocoSource

Values:

| **enumerator** LocoSourceRoster

| **enumerator** LocoSourceEntry

enum Facing

Values:

| **enumerator** FacingForward

| **enumerator** FacingReversed

Variables

static const int MAX_FUNCTIONS = 32

const int MAX_OBJECT_NAME_LENGTH = 30

file DCCExProtocol.cpp

#include "DCCExProtocol.h"

Variables

static const int MIN_SPEED = 0

static const int MAX_SPEED = 126

file DCCExProtocol.h

#include "DCCExCSConsist.h"

#include "DCCExInbound.h"

#include "DCCExLoco.h"

#include "DCCExProtocolVersion.h"

#include "DCCExRoutes.h"

#include "DCCExTurnouts.h"

#include "DCCExTurntables.h"

```
#include <Arduino.h>
```

Enums

enum TrackPower

Values:

| *enumerator* **PowerOff**

| *enumerator* **PowerOn**

| *enumerator* **PowerUnknown**

enum TrackManagerMode

Values:

| *enumerator* **MAIN**

| *enumerator* **PROG**

| *enumerator* **DC**

| *enumerator* **DCX**

| *enumerator* **NONE**

enum MomentumAlgorithm

Values:

| *enumerator* **Linear**

| *enumerator* **Power**

enum JMRISensorState

Values:

| *enumerator* **Activated**

| *enumerator* **Deactivated**

Variables

```
const int MAX_OUTBOUND_COMMAND_LENGTH = 100
```

file DCCExProtocolVersion.h

Defines

DCCEX_PROTOCOL_VERSION

file DCCEXRoutes.cpp

```
#include "DCCEXRoutes.h"  
#include <Arduino.h>
```

file DCCEXRoutes.h

```
#include <Arduino.h>
```

Enums

enum RouteType

Values:

| **enumerator RouteTypeRoute**

| **enumerator RouteTypeAutomation**

file DCCEXTurnouts.cpp

```
#include "DCCEXTurnouts.h"  
#include <Arduino.h>
```

file DCCEXTurnouts.h

```
#include <Arduino.h>
```

file DCCEXTurntables.cpp

```
#include "DCCEXTurntables.h"  
#include <Arduino.h>
```

file DCCEXTurntables.h

```
#include <Arduino.h>
```

Enums

enum TurntableType

Values:

| **enumerator TurntableTypeDCC**

| **enumerator TurntableTypeEXTT**

| **enumerator TurntableTypeUnknown**

dir /home/runner/work/DCCEXProtocol/DCCEXProtocol/src